Azure Data Analytics Case Study

1. Introduction

Modern transportation and mobility systems generate large volumes of trip data containing information about travel time, distance, purpose, and location. Analyzing this data can provide valuable insights into travel behavior, operational efficiency, and usage patterns.

Traditional data processing tools struggle to handle large datasets efficiently. Big data technologies enable scalable processing and analysis of such datasets.

This case study focuses on analyzing trip data using Apache Spark with PySpark on cloud infrastructure provided by Microsoft Azure. The analysis aims to identify travel patterns, evaluate efficiency metrics, detect anomalies, and derive insights from the dataset.

The system uses distributed processing to handle large datasets efficiently and uses Python visualization libraries to present analytical insights.

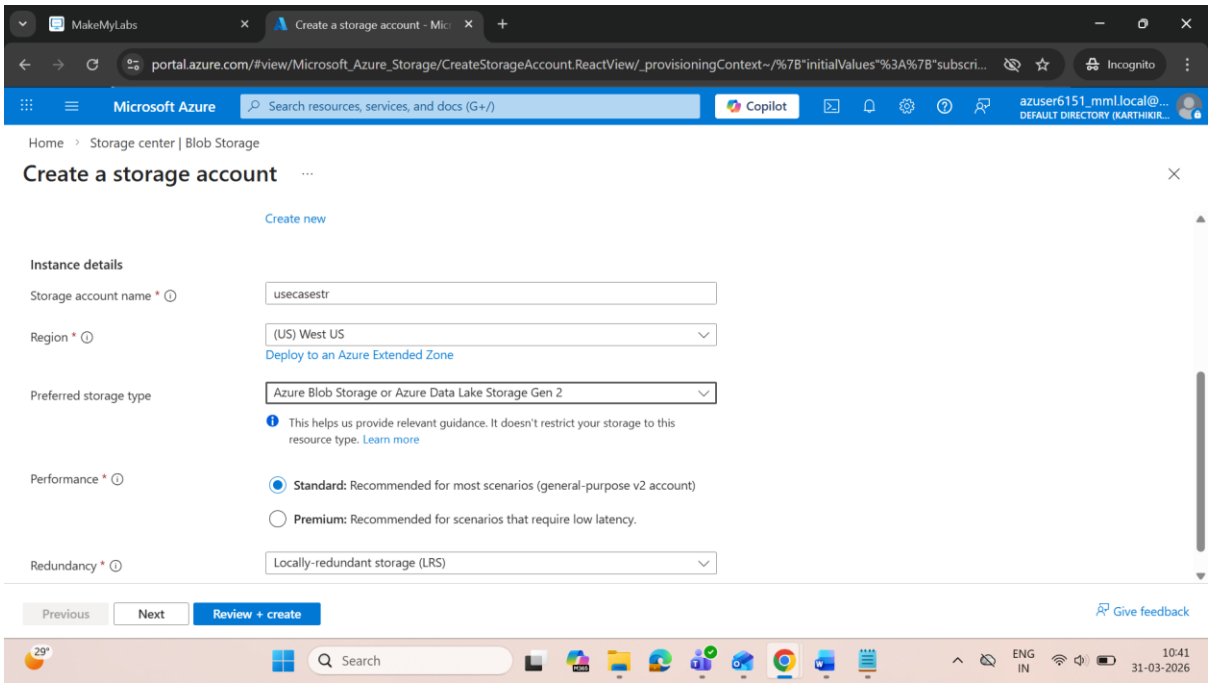
2. System Architecture

The big data analytics workflow implemented in Azure follows the architecture below.

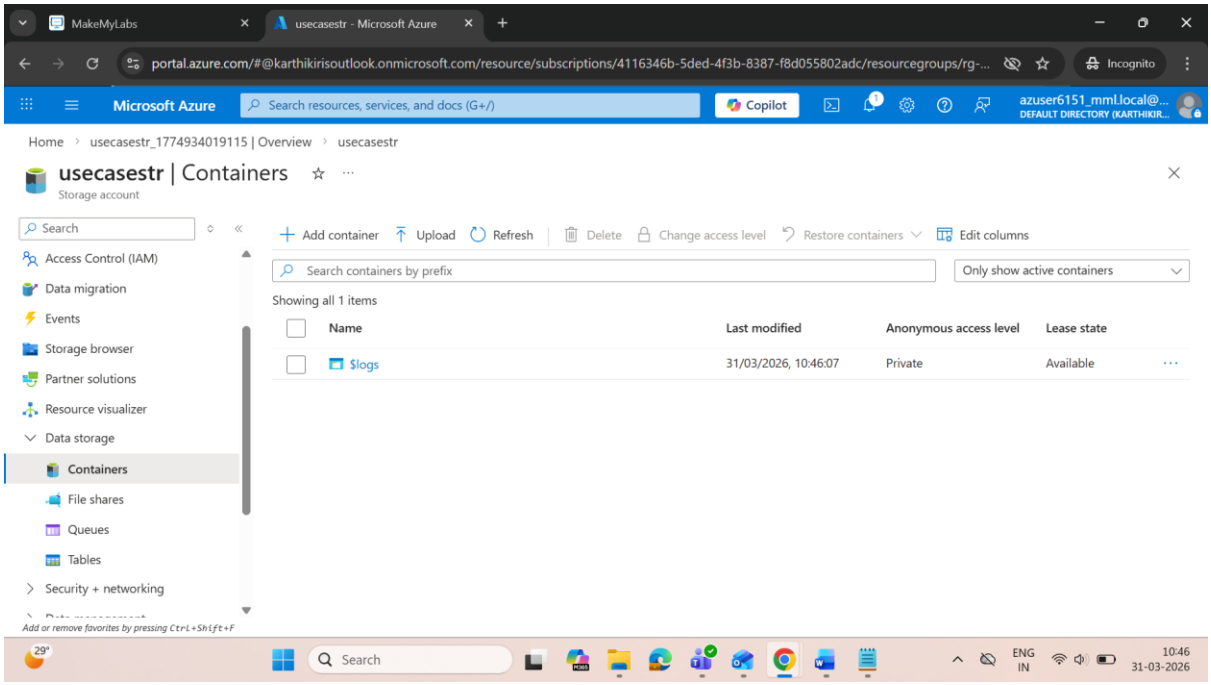
1. The raw dataset is stored in Azure Data Lake Storage, which provides scalable storage for large datasets.
2. Data processing and transformation are performed using Azure Databricks with Apache Spark and PySpark.
3. Spark performs distributed processing to clean, transform, and analyze the dataset.
4. Aggregated results are generated using Spark SQL and PySpark transformations.
5. Analytical insights are visualized using Python libraries such as Matplotlib and Seaborn.

This architecture allows efficient processing of large-scale datasets and supports scalable analytical workflows.

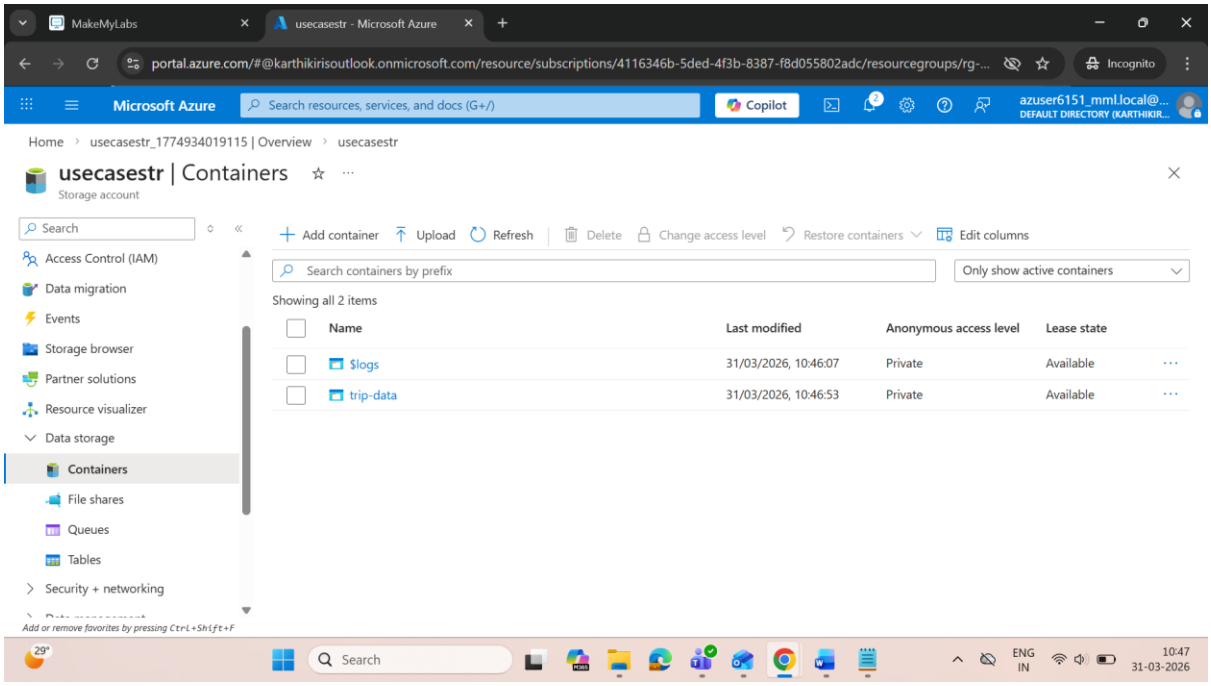
Step1:Create storage account, Fill the required details and click review+create

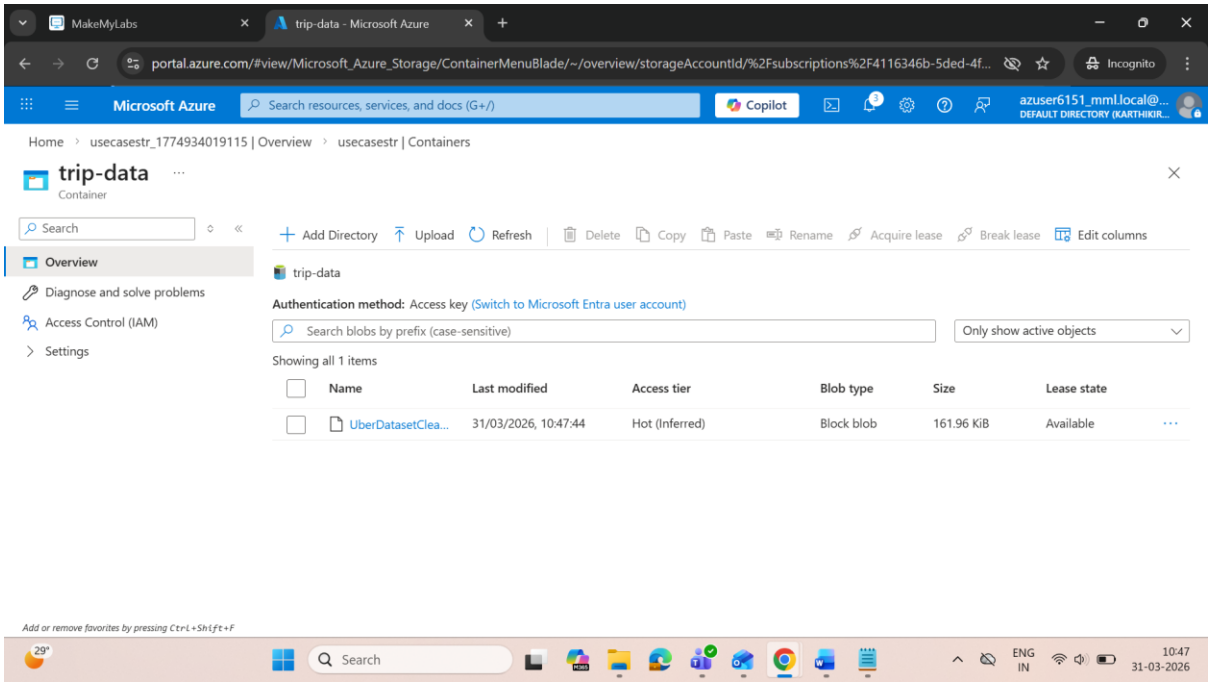


Step2: once it is deployed Go to resources,click storage account and Click Add container

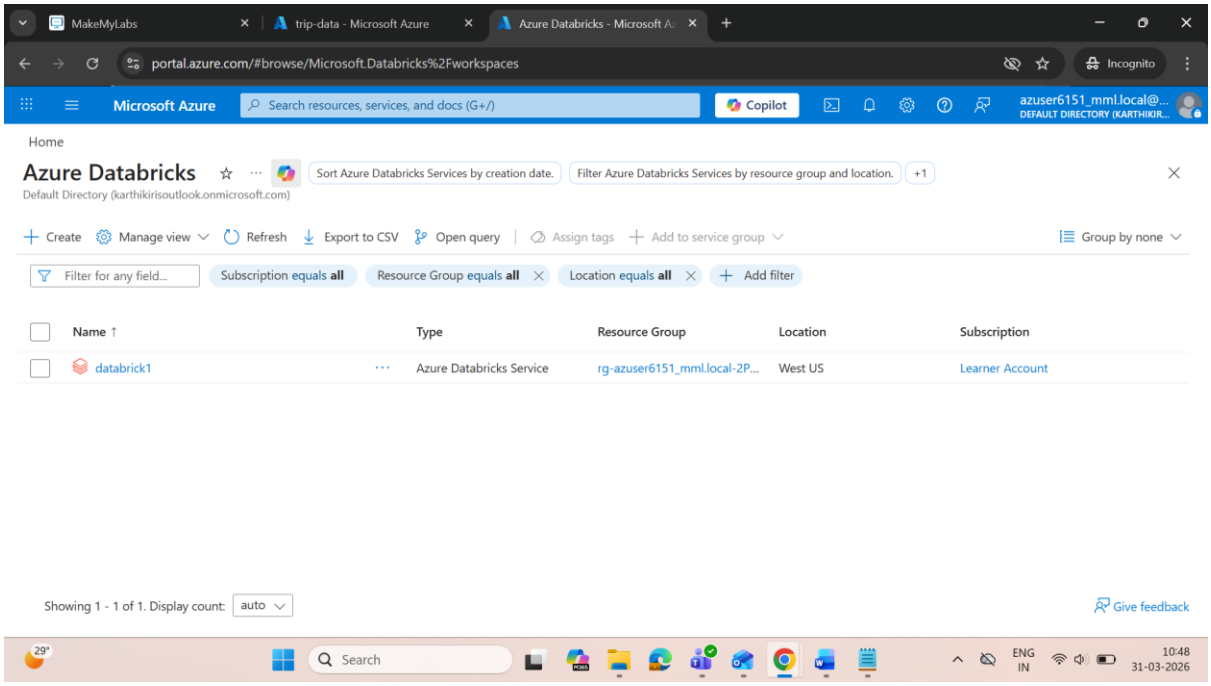


New container is created.open that and upload csv file

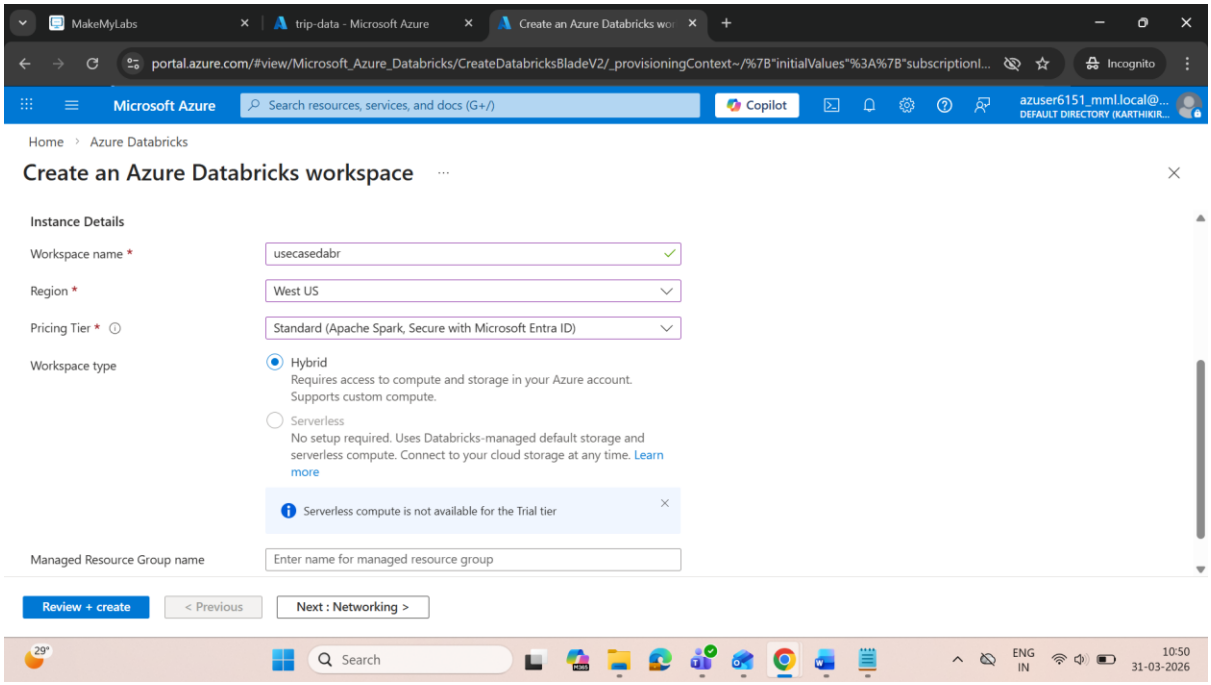




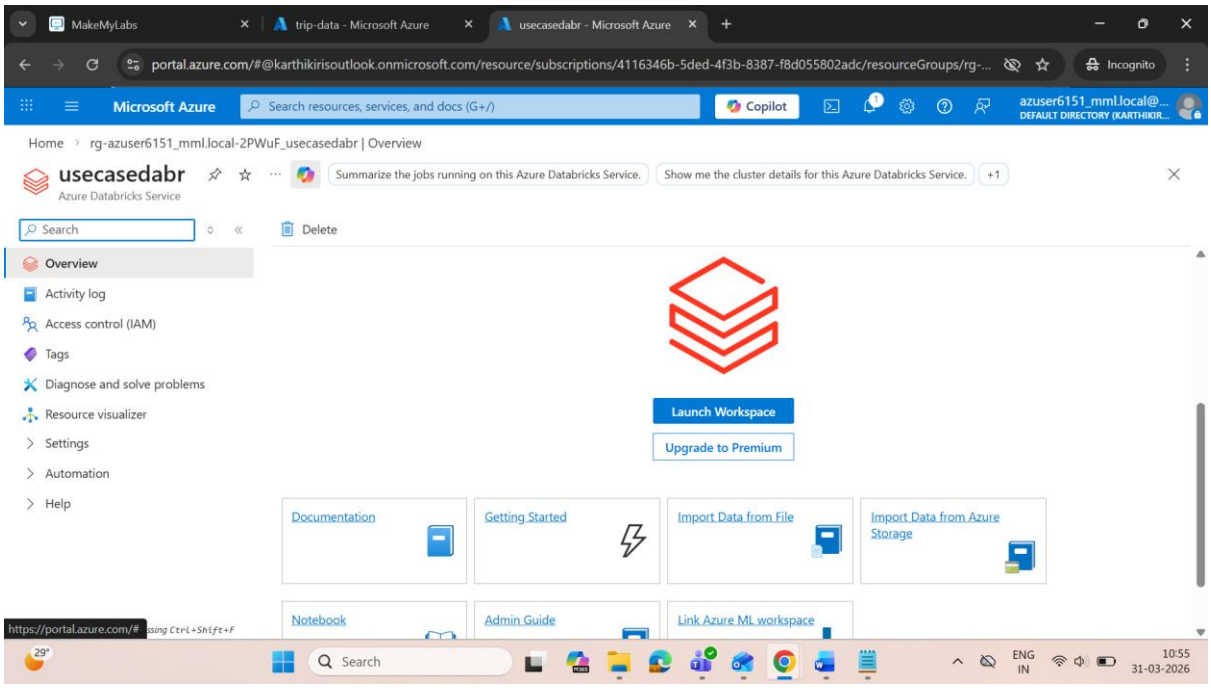
Step3:Now create Databrick account. Open databrick and click create



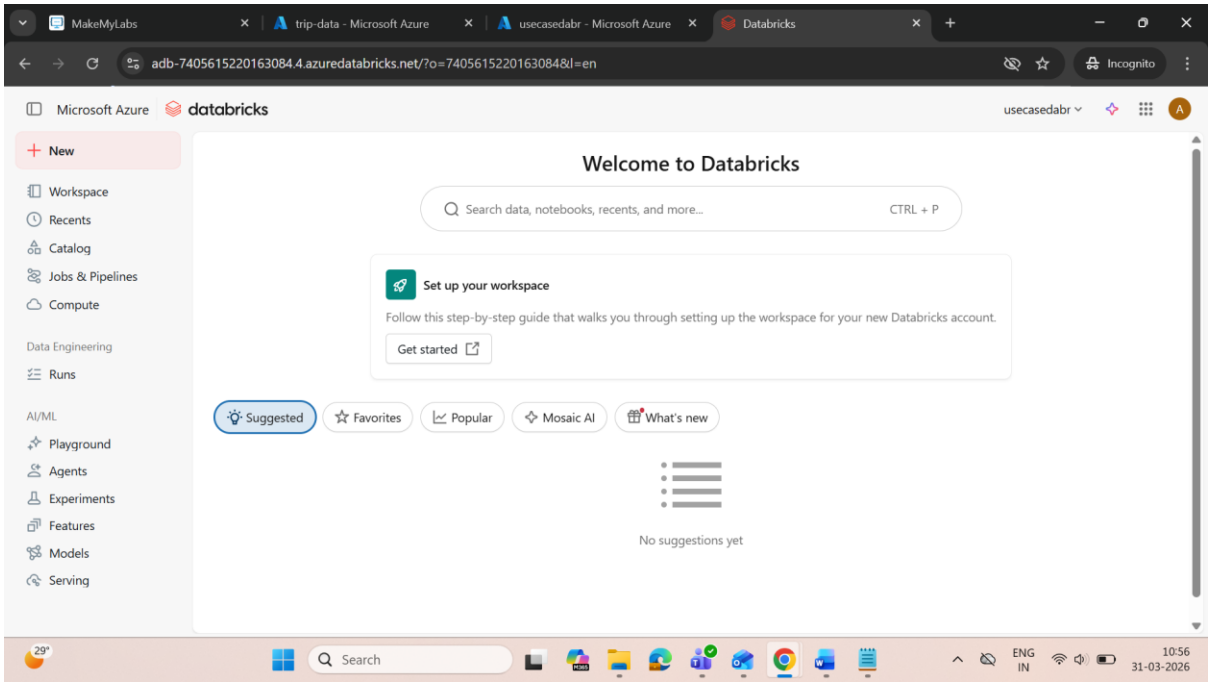
Fill the required details and click review+create



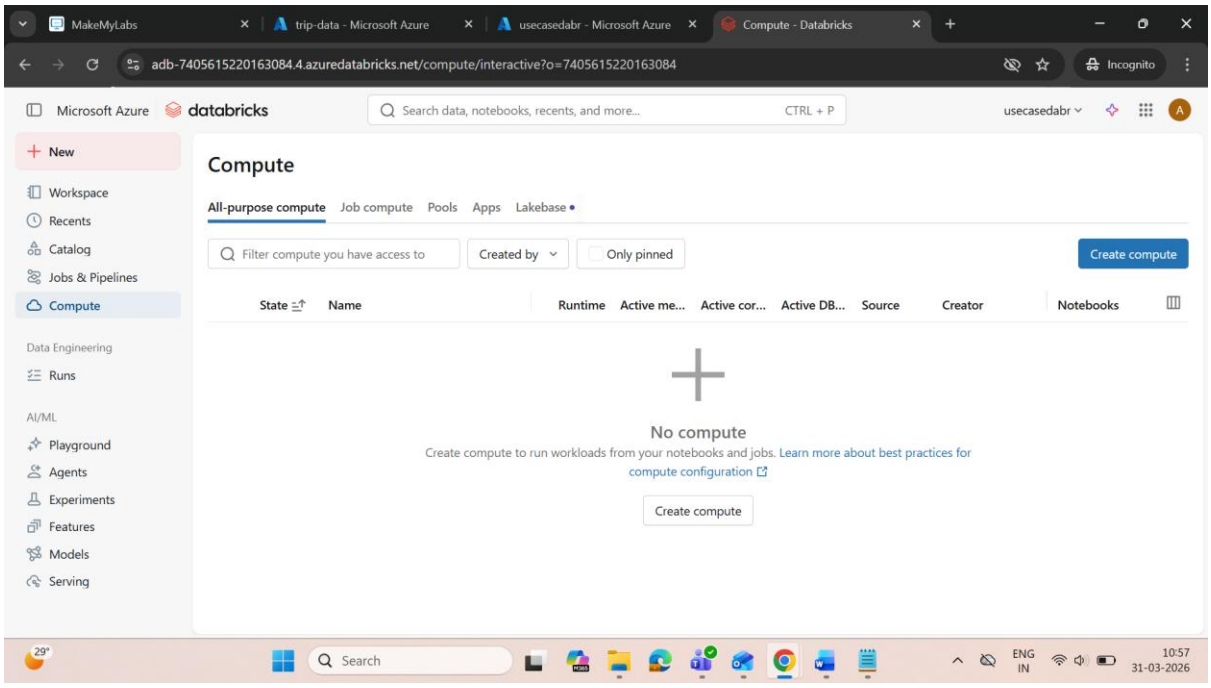
Once it is deployed go to resource, that opens as below picture. Now click launch workspace



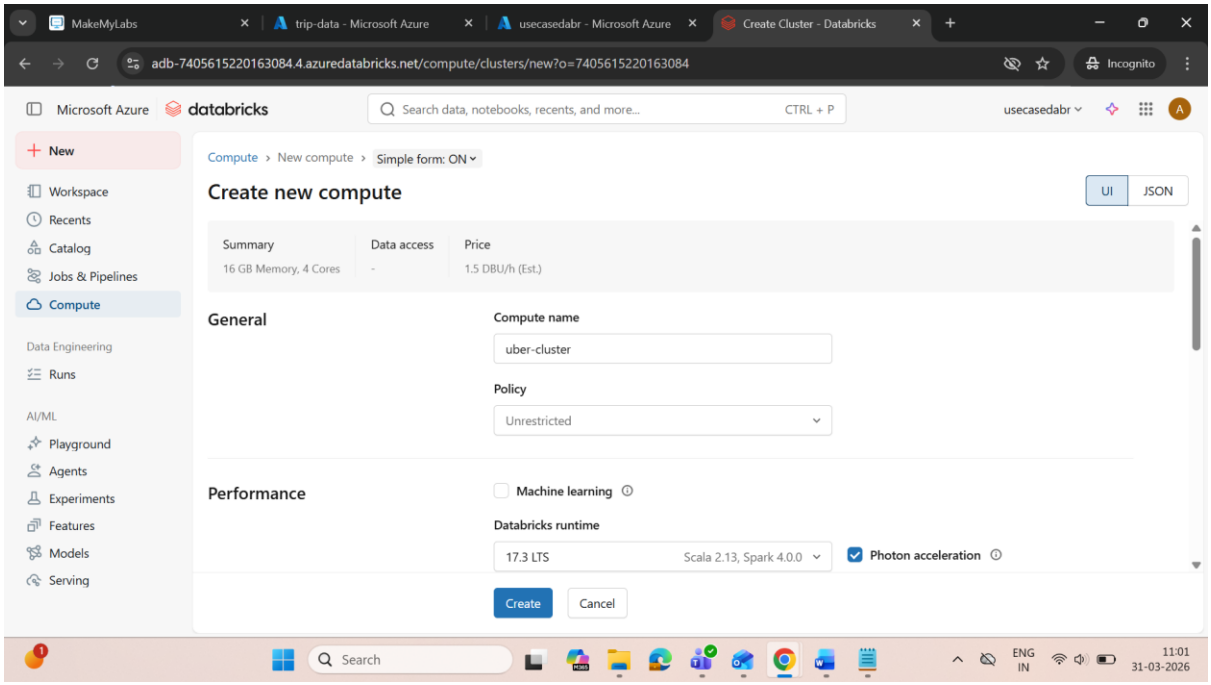
Click compute

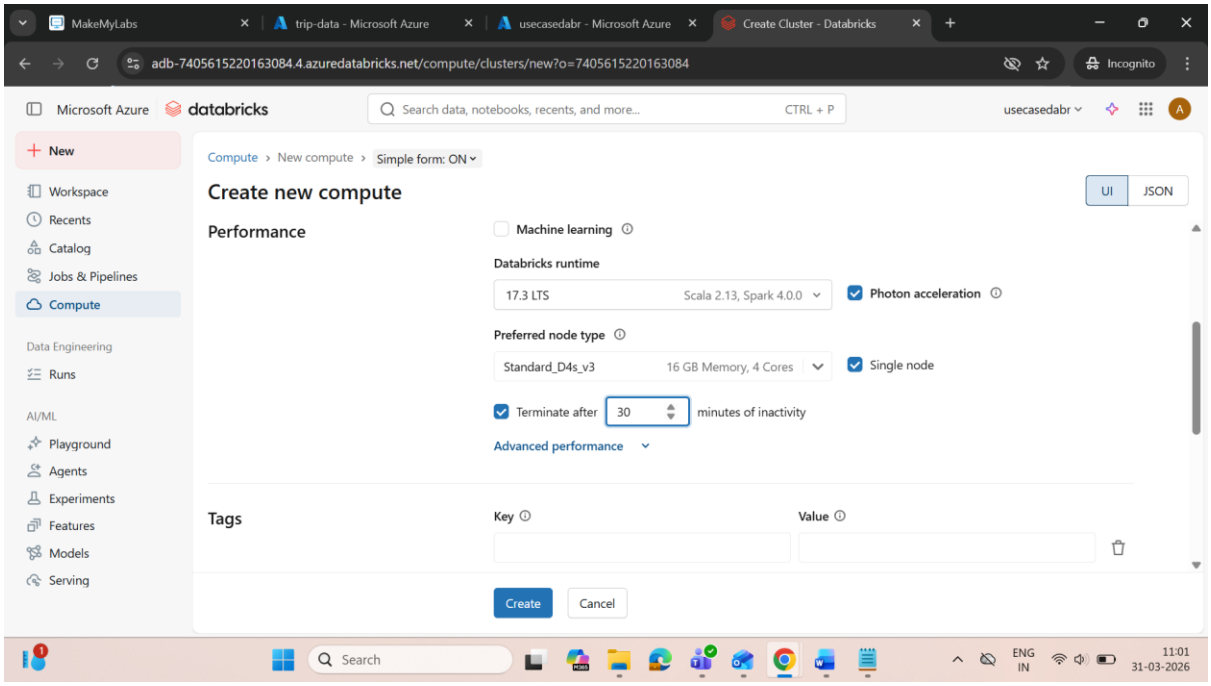


Click create compute

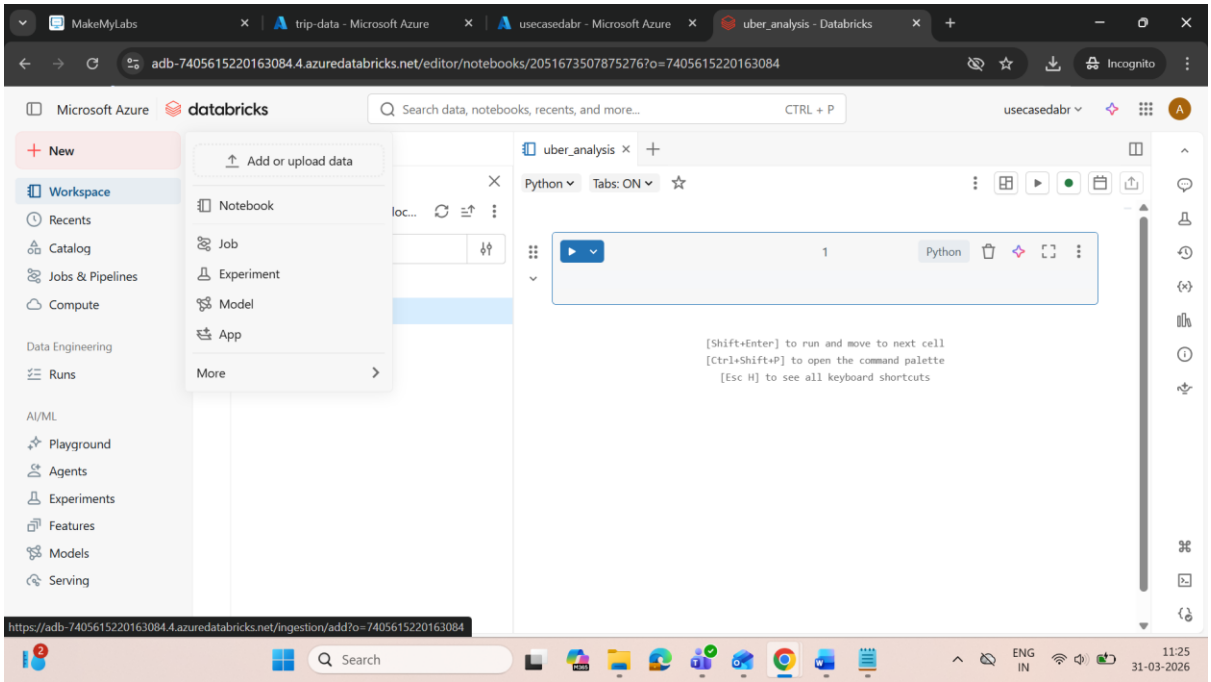


Fill the required details and click create

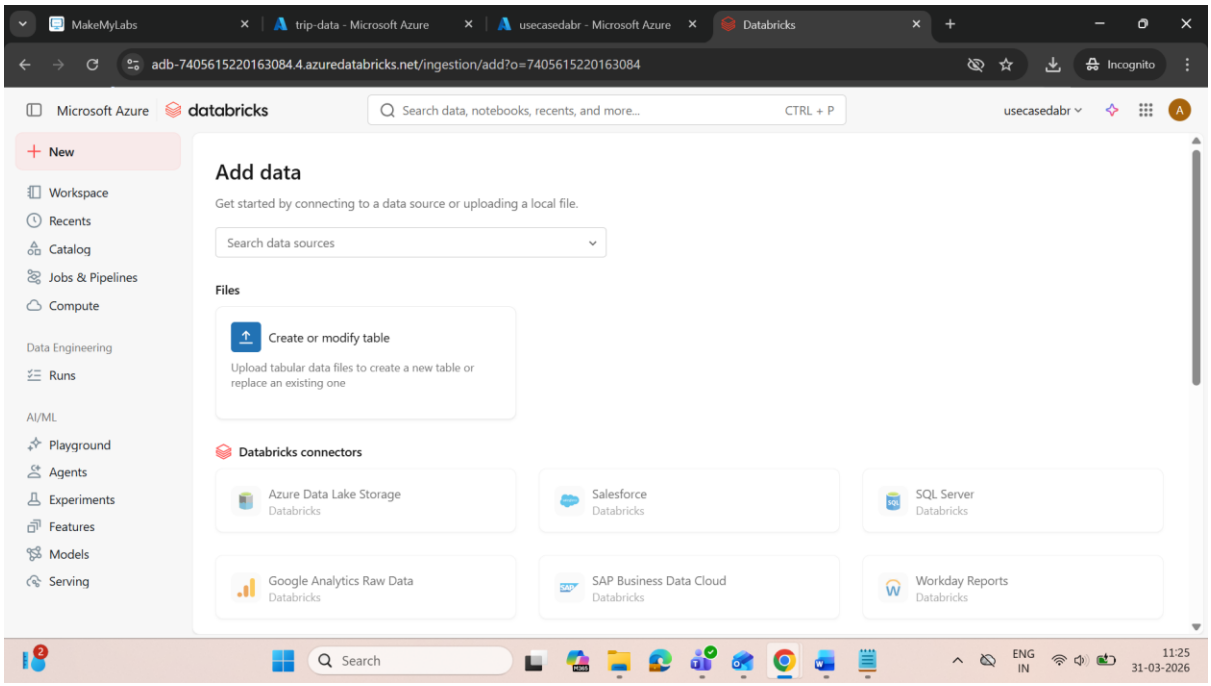




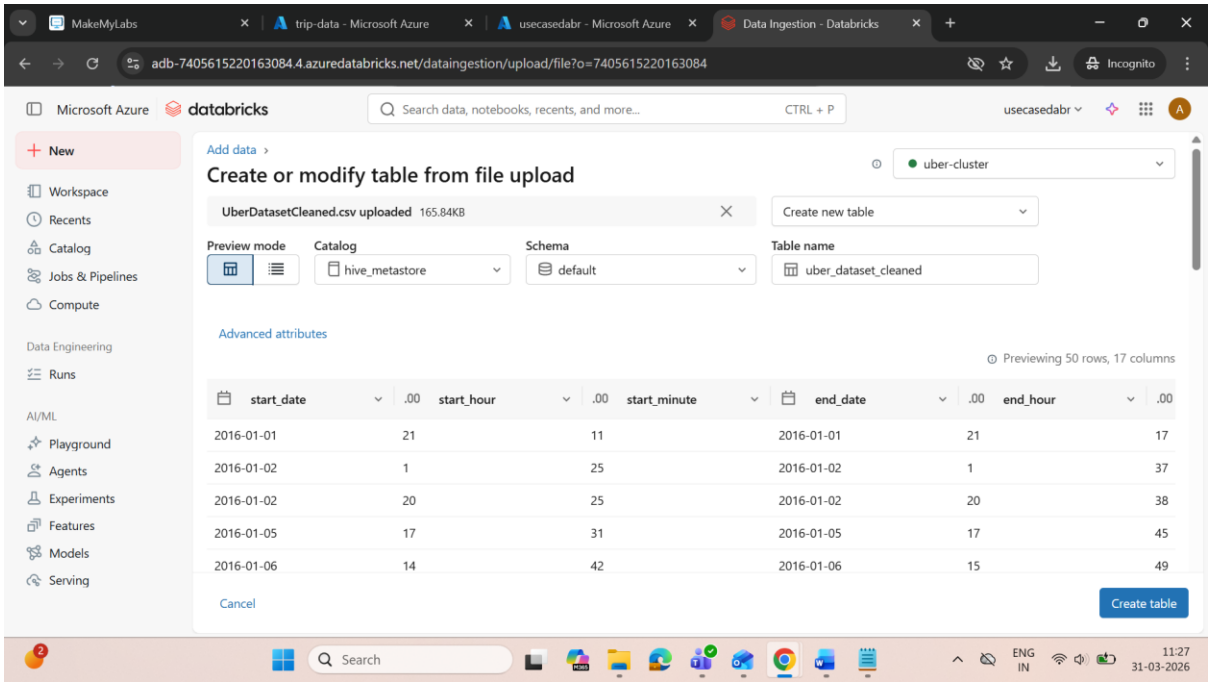
Click new -> Add or upload data



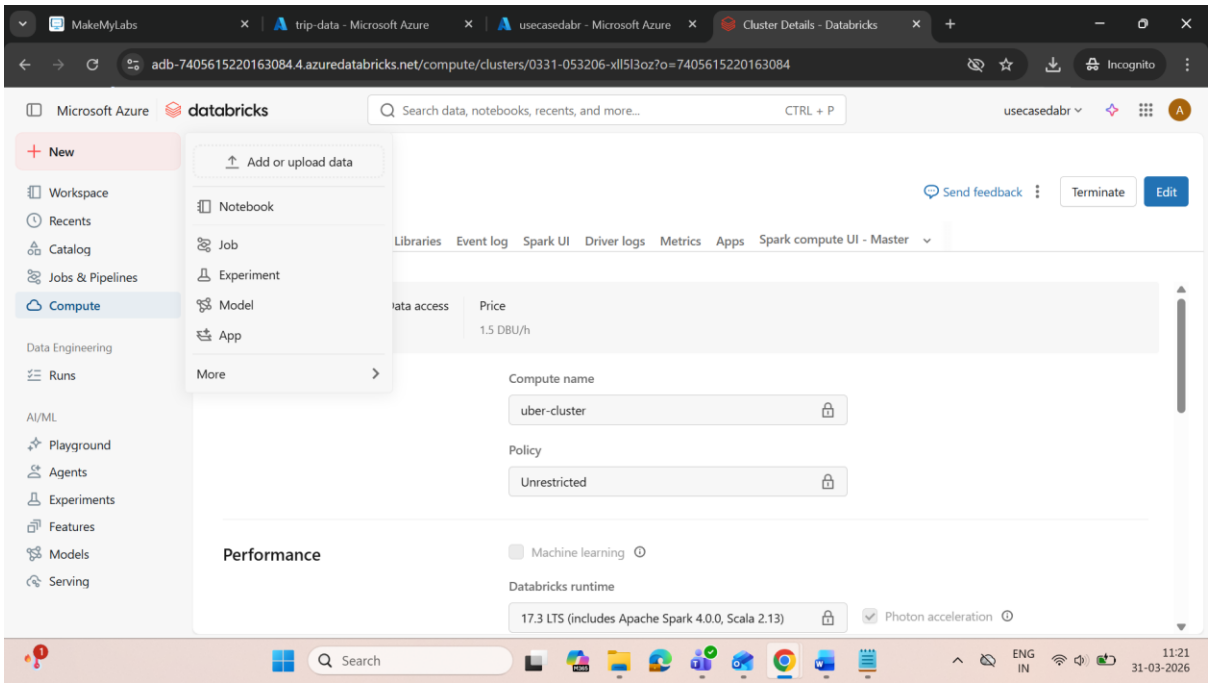
Click create or modify data



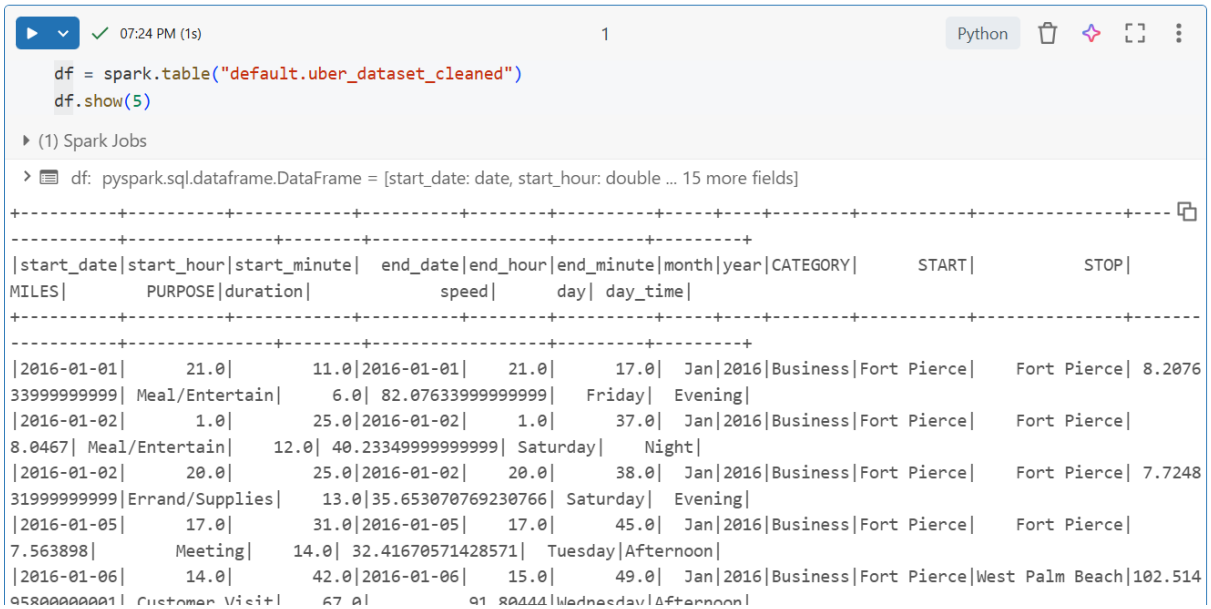
Click create table



Click new -> notebook and connect cluster



Load the dataset



07:24 PM (1s) 2 Python

```python
print("Total Rows:", df.count())
print("Total Columns:", len(df.columns))
print("Columns:", df.columns)
df.printSchema()
```

(2) Spark Jobs

```
Columns: ['start_date', 'start_hour', 'start_minute', 'end_date', 'end_hour', 'end_minute', 'month', 'year', 'CATEGORY', 'START', 'STOP', 'MILES', 'PURPOSE', 'duration', 'speed', 'day', 'day_time']
root
 |-- start_date: date (nullable = true)
 |-- start_hour: double (nullable = true)
 |-- start_minute: double (nullable = true)
 |-- end_date: date (nullable = true)
 |-- end_hour: double (nullable = true)
 |-- end_minute: double (nullable = true)
 |-- month: string (nullable = true)
 |-- year: long (nullable = true)
 |-- CATEGORY: string (nullable = true)
 |-- START: string (nullable = true)
 |-- STOP: string (nullable = true)
 |-- MILES: double (nullable = true)
 |-- PURPOSE: string (nullable = true)
 |-- duration: double (nullable = true)
 |-- speed: string (nullable = true)
 |-- day: string (nullable = true)
```

07:24 PM (<1s) 3

```python
df = df.dropDuplicates()
```

df: pyspark.sql.dataframe.DataFrame = [start_date: date, start_hour: double ... 15 more fields]

07:24 PM (1s) 4 Python

```python
from pyspark.sql.functions import col, sum as spark_sum, when

df.select([
    spark_sum(when(col(c).isNull(), 1).otherwise(0)).alias(c)
    for c in df.columns
]).show()
```

(3) Spark Jobs

```
+----------+----------+------------+--------+--------+----------+-----+----+--------+-----+----+-----+-------+--------+-----+---+--------+
|start_date|start_hour|start_minute|end_date|end_hour|end_minute|month|year|CATEGORY|START|STOP|MILES|PURPOSE|duration|speed|day|day_time|
+----------+----------+------------+--------+--------+----------+-----+----+--------+-----+----+-----+-------+--------+-----+---+--------+
|         0|         0|           0|       0|       0|         0|    0|   0|       0|    0|   0|    0|      0|       0|    0|  0|       0|
+----------+----------+------------+--------+--------+----------+-----+----+--------+-----+----+-----+-------+--------+-----+---+--------+
```

07:24 PM (<1s) 5 Python

```python
df = df.fillna({
    "PURPOSE": "Unknown",
    "CATEGORY": "Unknown"
})
```

df: pyspark.sql.dataframe.DataFrame = [start_date: date, start_hour: double ... 15 more fields]

07:24 PM (<1s) 6

```python
from pyspark.sql.functions import col

df = df.withColumn("MILES", col("MILES").cast("double")) \
        .withColumn("duration", col("duration").cast("double")) \
        .withColumn("speed", col("speed").cast("double")) \
        .withColumn("start_hour", col("start_hour").cast("int")) \
        .withColumn("start_minute", col("start_minute").cast("int")) \
        .withColumn("end_hour", col("end_hour").cast("int")) \
        .withColumn("end_minute", col("end_minute").cast("int")) \
        .withColumn("year", col("year").cast("int"))
```

df: pyspark.sql.dataframe.DataFrame = [start_date: date, start_hour: integer ... 15 more fields]

4.1 Trip Duration Analysis

- Calculate average trip duration
- Identify trips with unusually long or short durations
- Compare duration across different trip categories

07:24 PM (3s) 8 Python

```python
from pyspark.sql.functions import *

df.select(
    avg("duration").alias("avg_duration"),
    max("duration").alias("max_duration"),
    min("duration").alias("min_duration")
).show()


df.groupBy("CATEGORY").agg(
    avg("duration").alias("avg_duration")
).show()

df.filter(df.duration > 60).select(
    "START", "STOP", "CATEGORY", "MILES", "duration", "PURPOSE"
).show(20, truncate=False)

df.filter(df.duration < 5).select(
    "START", "STOP", "CATEGORY", "MILES", "duration", "PURPOSE"
).show(20, truncate=False)
```

(10) Spark Jobs

```
+-----------------+------------+------------+
|     avg_duration|max_duration|min_duration|
+-----------------+------------+------------+
|22.67719298245614|       330.0|         0.0|
+-----------------+------------+------------+

+--------+------------------+
|CATEGORY|      avg_duration|
+--------+------------------+
|Personal|20.466666666666665|
|Business|22.832863849765257|
+--------+------------------+

+----------------+----------------+--------+------------------+--------+---------------+
|START           |STOP            |CATEGORY|MILES             |duration|PURPOSE        |
+----------------+----------------+--------+------------------+--------+---------------+
|Mebane          |Cary            |Business|72.742168         |73.0    |Meeting        |
|Newland         |Boone           |Personal|67.43134599999999 |88.0    |Errand/Supplies|
|Lahore          |Unknown Location|Business|11.748182         |80.0    |Temporary Site |
|Unknown Location|Unknown Location|Business|154.818508        |144.0   |Airport/Travel |
|Unknown Location|Unknown Location|Business|30.57746          |77.0    |Airport/Travel |
```

4.2 Distance and Speed Analysis

- Calculate average speed for different trip categories
- Identify unrealistic speed values
- Evaluate travel efficiency

07:24 PM (5s) 9 Python

```python
# Average distance and speed
df.select(
    avg("MILES").alias("avg_miles"),
    avg("speed").alias("avg_speed")
).show()
# By category
df.groupBy("CATEGORY").agg(
    avg("MILES").alias("avg_miles"),
    avg("speed").alias("avg_speed")
).show()
# Unrealistic speeds
df.filter((df.speed > 120) | (df.speed < 1)).show()
# Highest speed trips
df.orderBy(df.speed.desc()).show(10)
# Lowest speed trips
df.orderBy(df.speed.asc()).show(10)
# Efficiency
df = df.withColumn(
    "duration_per_mile",
    when(df.MILES > 0, df.duration / df.MILES).otherwise(None)
)

df.select(
    avg("duration_per_mile").alias("avg_duration_per_mile")).show()
```

```
+------------------+---------+
|         avg_miles|avg_speed|
+------------------+---------+
|16.296120371929877| Infinity|
+------------------+---------+

+--------+------------------+-----------------+
|CATEGORY|         avg_miles|        avg_speed|
+--------+------------------+-----------------+
|Personal|15.155691226666669|33.66036530434004|
|Business| 16.37643228356813|         Infinity|
+--------+------------------+-----------------+

+----------+----------+------------+----------+--------+----------+-----+----+--------+----------------+----------------+----------------
---+--------------+--------+------------------+---------+---------+
|start_date|start_hour|start_minute|  end_date|end_hour|end_minute|month|year|CATEGORY|           START|            STOP|             MI
LES|       PURPOSE|duration|             speed|      day| day_time|
+----------+----------+------------+----------+--------+----------+-----+----+--------+----------------+----------------+----------------
---+--------------+--------+------------------+---------+---------+
|2016-10-06|        18|          37|2016-10-06|      18|        39|  Oct|2016|Business|Unknown Location|Unknown Location|29.611855999999
996|Temporary Site|     2.0| 888.3556799999999| Thursday|Afternoon|
```

4.3 Time-of-Day Travel Pattern Analysis

- Identify peak travel hours
- Analyze trip frequency during different time periods
- Compare travel activity across morning, afternoon, evening, and night

07:24 PM (3s) 10

```python
# 4.3 Time-of-Day Travel Pattern Analysis

# Trips per hour
df.groupBy("start_hour").count().orderBy("start_hour").show(24)

# Time segments
df.groupBy("day_time").count().orderBy("count", ascending=False).show()

# Peak hours
df.groupBy("start_hour").count().orderBy("count", ascending=False).show(5)
```

(9) Spark Jobs

```
+---------+-----+
| day_time|count|
+---------+-----+
|Afternoon|  558|
|  Morning|  313|
|  Evening|  232|
|    Night|   37|
+---------+-----+

+----------+-----+
|start_hour|count|
+----------+-----+
|        15|   98|
|        17|   95|
|        13|   94|
|        18|   94|
|        14|   89|
+----------+-----+
only showing top 5 rows
```

```
+----------+-----+
|start_hour|count|
+----------+-----+
|         0|   19|
|         1|    5|
|         2|    2|
|         3|    3|
|         5|    4|
|         6|    4|
|         7|   13|
|         8|   35|
|         9|   51|
|        10|   65|
|        11|   72|
|        12|   77|
|        13|   94|
|        14|   89|
|        15|   98|
|        16|   88|
|        17|   95|
|        18|   94|
```

4.4 Day-wise Travel Pattern Analysis

- Identify the busiest days for travel
- Compare weekday and weekend travel behavior
- Measure total distance travelled per day

07:24 PM (2s) 11

```python
#Day-wise Travel Pattern Analysis

from pyspark.sql.functions import sum as spark_sum, avg

# Trips per day
df.groupBy("day").count().orderBy("count", ascending=False).show()

# Distance per day
df.groupBy("day").agg(
    spark_sum("MILES").alias("total_miles")
).orderBy("total_miles", ascending=False).show()

# Avg duration per day
df.groupBy("day").agg(
    avg("duration").alias("avg_duration")
).orderBy("avg_duration", ascending=False).show()
```

(9) Spark Jobs

```
+----------+-----+
```

```
+---------+-----+
|      day|count|
+---------+-----+
|   Friday|  202|
|   Monday|  174|
|  Tuesday|  172|
| Thursday|  153|
|   Sunday|  147|
|Wednesday|  146|
| Saturday|  146|
+---------+-----+

+---------+------------------+
|      day|       total_miles|
+---------+------------------+
|   Friday| 3793.536247999996|
| Saturday|       2912.583532|
| Thursday|2905.0196339999998|
|   Sunday| 2714.151909999998|
|   Monday|2232.7983159999967|
|  Tuesday|2023.9059839999995|
```

4.5 Category-Based Trip Analysis

- Compare business and personal travel
- Analyze average distance and duration for each category
- Identify dominant travel purposes

07:24 PM (2s) 12

```python
# 4.5 Category-Based Trip Analysis


# 4.5.1 Compare business and personal travel (trip count)
df.groupBy("CATEGORY").count().show()

# 4.5.2 Average distance and duration by category
df.groupBy("CATEGORY").agg(
    avg("MILES").alias("avg_distance"),
    avg("duration").alias("avg_duration"),
    avg("speed").alias("avg_speed")
).show()

# 4.5.3 Identify dominant travel purposes
df.groupBy("PURPOSE").count() \
  .orderBy(col("count").desc()) \
  .show(10, truncate=False)
```

(9) Spark Jobs

```
+--------+-----+
|CATEGORY|count|
+--------+-----+
|Personal|   75|
|Business| 1065|
+--------+-----+

+--------+------------------+------------------+-----------------+
|CATEGORY|      avg_distance|      avg_duration|        avg_speed|
+--------+------------------+------------------+-----------------+
|Personal|15.155691226666669|20.466666666666665|33.66036530434004|
|Business| 16.37643228356813|22.832863849765257|         Infinity|
+--------+------------------+------------------+-----------------+

+---------------+-----+
|PURPOSE        |count|
+---------------+-----+
|Meeting        |295  |
|Meal/Entertain |245  |
|Temporary Site |194  |
|Errand/Supplies|173  |
```

4.6 Monthly Travel Trend Analysis

- Calculate total trips per month
- Identify months with highest travel activity
- Analyze trends across years

07:24 PM (3s) 13

```python
#Monthly Travel Trend Analysis

#Total trips per month
df.groupBy("month").count().orderBy("month").show()

#Identify months with highest travel activity
df.groupBy("month").count() \
  .orderBy(col("count").desc()) \
  .show()

#Analyze trends across years
df.groupBy("year", "month").count() \
  .orderBy("year", "month") \
  .show()
```

(9) Spark Jobs

```
+-----+-----+
|month|count|
+-----+-----+
|April|   53|
|  Aug|  133|
|  Dec|  146|
|  Feb|  114|
|  Jan|   61|
| July|  109|
| June|  102|
|  Mar|  112|
|  May|   48|
|  Nov|  122|
|  Oct|  105|
|  Sep|   35|
+-----+-----+
```

```
|month|count|
+-----+-----+
|  Dec|  146|
|  Aug|  133|
|  Nov|  122|
|  Feb|  114|
|  Mar|  112|
| July|  109|
|  Oct|  105|
| June|  102|
|  Jan|   61|
|April|   53|
|  May|   48|
|  Sep|   35|
+-----+-----+

+----+-----+-----+
|year|month|count|
+----+-----+-----+
|2016|April|   53|
|2016|  Aug|  133|
```

4.7 Travel Efficiency Analysis

- Miles travelled per trip
- Duration per mile
- Average speed

07:24 PM (3s) 14 Python

```python
#Travel Efficiency Analysis
# Create duration_per_mile safely
df_eff = df.withColumn(
    "duration_per_mile",
    when((col("MILES") > 0) & col("duration").isNotNull(), col("duration") / col("MILES"))
    .otherwise(None)
)
#Overall efficiency metrics
df_eff.select(
    avg("MILES").alias("avg_miles_per_trip"),
    avg("duration_per_mile").alias("avg_duration_per_mile"),
    avg("speed").alias("avg_speed")
).show()
#Efficiency by category
df_eff.groupBy("CATEGORY").agg(
    avg("MILES").alias("avg_miles_per_trip"),
    avg("duration_per_mile").alias("avg_duration_per_mile"),
    avg("speed").alias("avg_speed")
).show()
#Least efficient trips (high duration per mile)
df_eff.select(
    "START", "STOP", "CATEGORY", "MILES", "duration", "speed", "duration_per_mile", "PURPOSE"
).orderBy(col("duration_per_mile").desc()).show(10, truncate=False)
#Most efficient trips (low duration per mile)
df_eff.select(
    "START", "STOP", "CATEGORY", "MILES", "duration", "speed", "duration_per_mile", "PURPOSE"
).orderBy(col("duration_per_mile").asc()).show(10, truncate=False)
```

df_eff: pyspark.sql.dataframe.DataFrame = [start_date: date, start_hour: integer ... 16 more fields]

```
+------------------+---------------------+---------+
|avg_miles_per_trip|avg_duration_per_mile|avg_speed|
+------------------+---------------------+---------+
|16.296120371929877|   2.0746203520810544| Infinity|
+------------------+---------------------+---------+

+--------+------------------+---------------------+-----------------+
|CATEGORY|avg_miles_per_trip|avg_duration_per_mile|        avg_speed|
+--------+------------------+---------------------+-----------------+
|Personal|15.155691226666669|   2.1474954446884365|33.66036530434004|
|Business| 16.37643228356813|    2.069488303305887|         Infinity|
+--------+------------------+---------------------+-----------------+

+------------------------+----------------+--------+------------------+--------+------------------+------------------+---------------+
|START                   |STOP            |CATEGORY|MILES             |duration|speed             |duration_per_mile |PURPOSE        |
+------------------------+----------------+--------+------------------+--------+------------------+------------------+---------------+
|Unknown Location        |Unknown Location|Business|12.713786         |121.0   |6.304356694214877 |9.517227991724887 |Airport/Travel |
|Unknown Location        |R?walpindi      |Business|15.449663999999999|120.0   |7.724831999999999 |7.767159208122585 |Temporary Site |
|Unknown Location        |Islamabad       |Business|20.277684         |154.0   |7.900396363636363 |7.594555670164304 |Temporary Site |
|Lahore                  |Unknown Location|Business|11.748182         |80.0    |8.8111365         |6.809564237258156 |Temporary Site |
|Kar?chi                 |Kar?chi         |Business|1.1265379999999998|7.0     |9.656039999999999 |6.213727366498069 |Meeting        |
```

4.8 Peak Travel Period Detection

- Count trips per hour
- Identify peak travel windows
- Analyze travel distribution across time segments

07:24 PM (2s) 15

```python
#Travel Period Detection
#Count trips per hour
df.groupBy("start_hour").count() \
  .orderBy("start_hour") \
  .show(24)

#Identify peak travel windows (top hours)
df.groupBy("start_hour").count() \
  .orderBy(col("count").desc()) \
  .show(10)

#Analyze travel distribution across time segments
df.groupBy("day_time").count() \
  .orderBy(col("count").desc()) \
  .show()
```

(9) Spark jobs

```
+----------+-----+
|start_hour|count|
+----------+-----+
|         0|   19|
|         1|    5|
|         2|    2|
|         3|    3|
|         5|    4|
|         6|    4|
|         7|   13|
|         8|   35|
|         9|   51|
|        10|   65|
|        11|   72|
|        12|   77|
|        13|   94|
|        14|   89|
|        15|   98|
|        16|   88|
|        17|   95|
|        18|   94|
```

```
|start_hour|count|
+----------+-----+
|        15|   98|
|        17|   95|
|        13|   94|
|        18|   94|
|        14|   89|
|        16|   88|
|        12|   77|
|        11|   72|
|        20|   71|
|        19|   68|
+----------+-----+
only showing top 10 rows
+---------+-----+
| day_time|count|
+---------+-----+
|Afternoon|  558|
|  Morning|  313|
|  Evening|  232|
|    Night|   37|
```

4.9 Long Distance Trip Identification

- Detect trips exceeding predefined distance thresholds
- Identify trips with unusually long duration
- Analyze high-resource travel patterns

07:24 PM (3s) 16 Pythor

```python
#Long Distance Trip Identification
#Detect trips exceeding predefined distance threshold (>20 miles)
df.filter(col("MILES") > 20).select(
    "START", "STOP", "CATEGORY", "MILES", "duration", "speed", "PURPOSE"
).show(20, truncate=False)
#Identify trips with unusually long duration (>60 minutes)
df.filter(col("duration") > 60).select(
    "START", "STOP", "CATEGORY", "MILES", "duration", "speed", "PURPOSE"
).show(20, truncate=False)
#Analyze high-resource travel patterns
df.filter((col("MILES") > 20) | (col("duration") > 60)).select(
    "START", "STOP", "CATEGORY", "MILES", "duration", "speed", "PURPOSE"
).show(20, truncate=False)
#Top 10 longest-distance trips
df.select(
    "START", "STOP", "CATEGORY", "MILES", "duration", "speed", "PURPOSE"
).orderBy(col("MILES").desc()).show(10, truncate=False)

#Top 10 longest-duration trips
df.select(
    "START", "STOP", "CATEGORY", "MILES", "duration", "speed", "PURPOSE"
).orderBy(col("duration").desc()).show(10, truncate=False)
```

```
+----------------+----------------+--------+------------------+--------+------------------+---------------+
|START           |STOP            |CATEGORY|MILES             |duration|speed             |PURPOSE        |
+----------------+----------------+--------+------------------+--------+------------------+---------------+
|Orlando         |Kissimmee       |Personal|22.208892000000002|17.0    |78.38432470588236 |Customer Visit |
|Cary            |Raleigh         |Business|21.887024         |39.0    |33.67234461538462 |Meeting        |
|Banner Elk      |Elk Park        |Personal|36.049215999999994|40.0    |54.073823999999995|Errand/Supplies|
|Cary            |Raleigh         |Business|25.266638         |29.0    |52.27580275862069 |Customer Visit |
|Capitol One     |Mcvan           |Business|23.174496         |15.0    |92.697984         |Meeting        |
|Houston         |Houston         |Business|35.244546         |36.0    |58.74091          |Customer Visit |
|New Orleans     |Kenner          |Business|20.760486         |17.0    |73.27230352941177 |Meal/Entertain |
|Colombo         |Katunayaka      |Business|34.439876         |44.0    |46.96346727272727 |Temporary Site |
|Unknown Location|Unknown Location|Business|29.611855999999996|2.0     |888.3556799999999 |Temporary Site |
|San Francisco   |Berkeley        |Business|21.243288         |40.0    |31.864932         |Between Offices|
|Unknown Location|Islamabad       |Business|20.438617999999998|42.0    |29.19802571428571 3|Temporary Site |
|Kenner          |New Orleans     |Business|21.243288         |19.0    |67.08406736842106 |Temporary Site |
|Unknown Location|Unknown Location|Business|51.98168199999999 |58.0    |53.77415379310344 |Meeting        |
|Durham          |Apex            |Business|24.622902         |35.0    |42.21068914285714 |Customer Visit |
|Mebane          |Cary            |Business|72.742168         |73.0    |59.78808328767124 |Meeting        |
|Unknown Location|Unknown Location|Business|40.555368         |51.0    |47.71219764705883 |Airport/Travel |
|Banner Elk      |Newland         |Personal|45.222454         |56.0    |48.45262928571428 |Errand/Supplies|
|R?walpindi      |Unknown Location|Business|30.094658         |29.0    |62.264809655172414|Airport/Travel |
```

4.10 Data Quality Monitoring

- Negative trip duration

- Missing values
- Unrealistic speeds
- Trips with zero distance but long duration
- End time earlier than start time

07:29 PM (3s) 17 Python

```python
df_q = df.withColumn("duration_num", expr("try_cast(duration as double)")) \
          .withColumn("speed_num", expr("try_cast(speed as double)")) \
          .withColumn("miles_num", expr("try_cast(MILES as double)")) \
          .withColumn("start_hour_num", expr("try_cast(start_hour as int)")) \
          .withColumn("start_minute_num", expr("try_cast(start_minute as int)")) \
          .withColumn("end_hour_num", expr("try_cast(end_hour as int)")) \
          .withColumn("end_minute_num", expr("try_cast(end_minute as int)"))
#Negative trip duration
print("Negative trip duration:")
df_q.filter(col("duration_num") < 0).show()
#Missing values in all columns
print("Missing values summary:")
df_q.select([spark_sum(when(col(c).isNull(), 1).otherwise(0)).alias(c) for c in df_q.columns]).show()
#Unrealistic speeds (>120)
print("Unrealistic speed values:")
df_q.filter(col("speed_num") > 120).show()
#Zero distance but long duration
print("Zero distance but long duration:")
df_q.filter((col("miles_num") == 0) & (col("duration_num") > 10)).show()
#End time earlier than start time
print("End time earlier than start time:")
df_q.filter((col("end_hour_num") < col("start_hour_num")) |
    ((col("end_hour_num") == col("start_hour_num")) & (col("end_minute_num") < col("start_minute_num")))
).show()
```

(6) Spark Jobs

df_q: pyspark.sql.dataframe.DataFrame = [start_date: date, start_hour: integer ... 23 more fields]

```
+----------+----------+------------+----------+--------+----------+-----+----+--------+----------------+----------------+----------------
---+--------------+--------+------------------+---------+---------+-------------------+------------+-----------------+----------------
-+--------------+----------------+------------+--------------+
|start_date|start_hour|start_minute|  end_date|end_hour|end_minute|month|year|CATEGORY|           START|            STOP|             MI
LES|       PURPOSE|duration|             speed|      day| day_time|  duration_per_mile|duration_num|         speed_num|         miles_nu
m|start_hour_num|start_minute_num|end_hour_num|end_minute_num|
+----------+----------+------------+----------+--------+----------+-----+----+--------+----------------+----------------+----------------
---+--------------+--------+------------------+---------+---------+-------------------+------------+-----------------+----------------
-+--------------+----------------+------------+--------------+
|2016-10-06|        18|          37|2016-10-06|      18|        39|  Oct|2016|Business|Unknown Location|Unknown Location|29.611855999999
996|Temporary Site|     2.0| 888.3556799999999| Thursday|Afternoon|0.06754051485323988|         2.0| 888.3556799999999|29.61185599999999
6|            18|              37|          18|            39|
|2016-12-22|        18|          38|2016-12-22|      18|        47|  Dec|2016|Business|Unknown Location|Unknown Location|         19.794
882|Temporary Site|     9.0|131.96588000000003| Thursday|Afternoon|0.45466297803644395|         9.0|131.96588000000003|         19.79488
2|            18|              38|          18|            47|
|2016-10-06|        17|          23|2016-10-06|      17|        40|  Oct|2016|Business|      R?walpindi|Unknown Location|        181.211
684|Temporary Site|    17.0| 639.5706494117646| Thursday|Afternoon|0.09381293537341666|        17.0| 639.5706494117646|        181.21168
4|            17|              23|          17|            40|
|2016-10-13|        13|           2|2016-10-13|      13|         2|  Oct|2016|Business|       Islamabad|       Islamabad|1.1265379999999
998|       Meeting|     0.0|          Infinity| Thursday|Afternoon|                0.0|         0.0|          Infinity|1.126537999999999
8|            13|               2|          13|             2|
```

4.11 Travel Behavior Segmentation

- Distance
- Duration
- Speed

- Time of day

07:35 PM (13s) 18 Python

```python
# 4.11 Travel Behavior Segmentation
from pyspark.sql.functions import expr, col
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.clustering import KMeans
# Step 1: Create safe numeric columns
df_ml = df.withColumn("MILES_num", expr("try_cast(MILES as double)")) \
           .withColumn("duration_num", expr("try_cast(duration as double)")) \
           .withColumn("speed_num", expr("try_cast(speed as double)")) \
           .withColumn("start_hour_num", expr("try_cast(start_hour as int)"))
# Step 2: Remove bad values
ml_df = df_ml.select("MILES_num", "duration_num", "speed_num", "start_hour_num") \
    .dropna() \
    .filter(
        (col("MILES_num").isNotNull()) &
        (col("duration_num").isNotNull()) &
        (col("speed_num").isNotNull()) &
        (col("start_hour_num").isNotNull()) &
        (col("speed_num") != float("inf")) &
        (col("speed_num") != float("-inf")) &
        (col("duration_num") > 0) &
        (col("MILES_num") > 0)
    )
```

```python
# Step 3: Assemble features
assembler = VectorAssembler(
    inputCols=["MILES_num", "duration_num", "speed_num", "start_hour_num"],outputCol="features"
)
feature_df = assembler.transform(ml_df)
# Step 4: Apply KMeans clustering
kmeans = KMeans(k=3, seed=1)
model = kmeans.fit(feature_df)
clustered_df = model.transform(feature_df)
# Step 5: Show results
clustered_df.select(
    "MILES_num", "duration_num", "speed_num", "start_hour_num", "prediction"
).show(20)
# Step 6: Cluster counts
clustered_df.groupBy("prediction").count().show()
```

(26) Spark Jobs

clustered_df: pyspark.sql.dataframe.DataFrame

df_ml: pyspark.sql.dataframe.DataFrame = [start_date: date, start_hour: integer ... 20 more fields]

feature_df: pyspark.sql.dataframe.DataFrame

ml_df: pyspark.sql.dataframe.DataFrame = [MILES_num: double, duration_num: double ... 2 more fields]

MILES_num	duration_num	speed_num	start_hour_num	prediction
22.208892000000002	17.0	78.38432470588236	23	0
21.887024	39.0	33.67234461538462	8	0
36.049215999999994	40.0	54.073823999999995	12	0
4.82802	15.0	19.31208	18	0
25.266638	29.0	52.27580275862069	13	0
1.9312079999999998	7.0	16.553211428571426	11	0
23.174496	15.0	92.697984	5	0
3.379614	10.0	20.277684	15	0
35.244546	36.0	58.74091	16	0
9.65604	15.0	38.62416	20	0
20.760486	17.0	73.27230352941177	19	0
2.735878	6.0	27.35878	17	0
11.587248	19.0	36.59130947368421	16	0
34.439876	44.0	46.96346727272727	15	0
7.0810960000000005	21.0	20.23170285714286	13	0
29.611855999999996	2.0	888.3556799999999	18	1
21.243288	40.0	31.864932	19	0
20.438617999999998	42.0	29.198025714285713	14	0

4.12 Travel Duration Prediction

- Distance travelled
- Time of day
- Day of the week
- Trip category

```
#Travel Duration Prediction
from pyspark.sql.functions import expr, col
from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler
from pyspark.ml.regression import LinearRegression
from pyspark.ml import Pipeline
from pyspark.ml.evaluation import RegressionEvaluator
# Step 1: Create safe numeric columns
df_pred = df.withColumn("MILES_num", expr("try_cast(MILES as double)")) \
            .withColumn("start_hour_num", expr("try_cast(start_hour as int)")) \
            .withColumn("duration_num", expr("try_cast(duration as double)"))
# Step 2: Select required columns and clean
data = df_pred.select(
    "MILES_num", "start_hour_num", "day", "CATEGORY", "duration_num"
).dropna()
# Keep only valid rows
data = data.filter(
    (col("MILES_num") > 0) &
    (col("duration_num") > 0) &
    (col("start_hour_num").between(0, 23))
)
# Step 3: Encode categorical columns
day_indexer = StringIndexer(inputCol="day", outputCol="day_index", handleInvalid="keep")
day_encoder = OneHotEncoder(inputCol="day_index", outputCol="day_vec")
cat_indexer = StringIndexer(inputCol="CATEGORY", outputCol="CATEGORY_index", handleInvalid="keep")
```

```python
# Step 4: Assemble features
assembler = VectorAssembler(
    inputCols=["MILES_num", "start_hour_num", "day_vec", "CATEGORY_vec"],
    outputCol="features"
)
# Step 5: Define regression model
lr = LinearRegression(featuresCol="features", labelCol="duration_num")
# Step 6: Build pipeline
pipeline = Pipeline(stages=[
    day_indexer, day_encoder,
    cat_indexer, cat_encoder,
    assembler, lr
])
# Step 7: Split data
train_data, test_data = data.randomSplit([0.8, 0.2], seed=42)
# Step 8: Train model
model = pipeline.fit(train_data)
# Step 9: Predict
predictions = model.transform(test_data)
# Step 10: Show prediction results
predictions.select(
    "MILES_num", "start_hour_num", "day", "CATEGORY", "duration_num", "prediction"
).show(20, truncate=False)
```

```python
# Step 11: Evaluate model
rmse_evaluator = RegressionEvaluator(
    labelCol="duration_num",
    predictionCol="prediction",
    metricName="rmse"
)

r2_evaluator = RegressionEvaluator(
    labelCol="duration_num",
    predictionCol="prediction",
    metricName="r2"
)

rmse = rmse_evaluator.evaluate(predictions)
r2 = r2_evaluator.evaluate(predictions)

print("RMSE:", rmse)
print("R2 Score:", r2)
```

(20) Spark Jobs

data: pyspark.sql.dataframe.DataFrame = [MILES_num: double, start_hour_num: integer ... 3 more fields]

df_pred: pyspark.sql.dataframe.DataFrame = [start_date: date, start_hour: integer ... 19 more fields]

predictions: pyspark.sql.dataframe.DataFrame

test_data: pyspark.sql.dataframe.DataFrame = [MILES_num: double, start_hour_num: integer ... 3 more fields]

```
+------------------+--------------+---------+--------+------------+------------------+
|MILES_num         |start_hour_num|day      |CATEGORY|duration_num|prediction        |
+------------------+--------------+---------+--------+------------+------------------+
|1.1265379999999998|1             |Saturday |Business|7.0         |14.744958660809209|
|1.1265379999999998|14            |Wednesday|Business|6.0         |12.004760160074982|
|1.1265379999999998|16            |Tuesday  |Business|6.0         |14.504499652579174|
|1.448406          |12            |Saturday |Business|5.0         |14.887788647114881|
|1.448406          |13            |Sunday   |Business|6.0         |11.060493493340449|
|1.448406          |15            |Wednesday|Business|6.0         |12.204657664872155|
|1.448406          |16            |Monday   |Business|8.0         |12.394363774559025|
|1.448406          |20            |Monday   |Business|4.0         |12.371536767162425|
|1.60934           |11            |Friday   |Personal|5.0         |12.320940490911669|
|1.60934           |18            |Wednesday|Business|1.0         |12.290339537647867|
|1.7702740000000001|7             |Friday   |Business|6.0         |13.787339967890592|
|1.7702740000000001|15            |Thursday |Business|4.0         |14.283256005003139|
|1.7702740000000001|17            |Tuesday  |Business|9.0         |14.910001414022668|
|1.9312079999999998|10            |Thursday |Business|11.0        |14.41459189257205 |
|1.9312079999999998|17            |Saturday |Business|8.0         |15.167661272838615|
|2.092142          |15            |Thursday |Business|11.0        |14.48886026164946 |
|2.2530759999999996|12            |Thursday |Business|8.0         |14.608782645520073|
|2.2530759999999996|16            |Tuesday  |Business|11.0        |15.224114550841303|
```

Visualization and Reporting

Trip Frequency by Hour

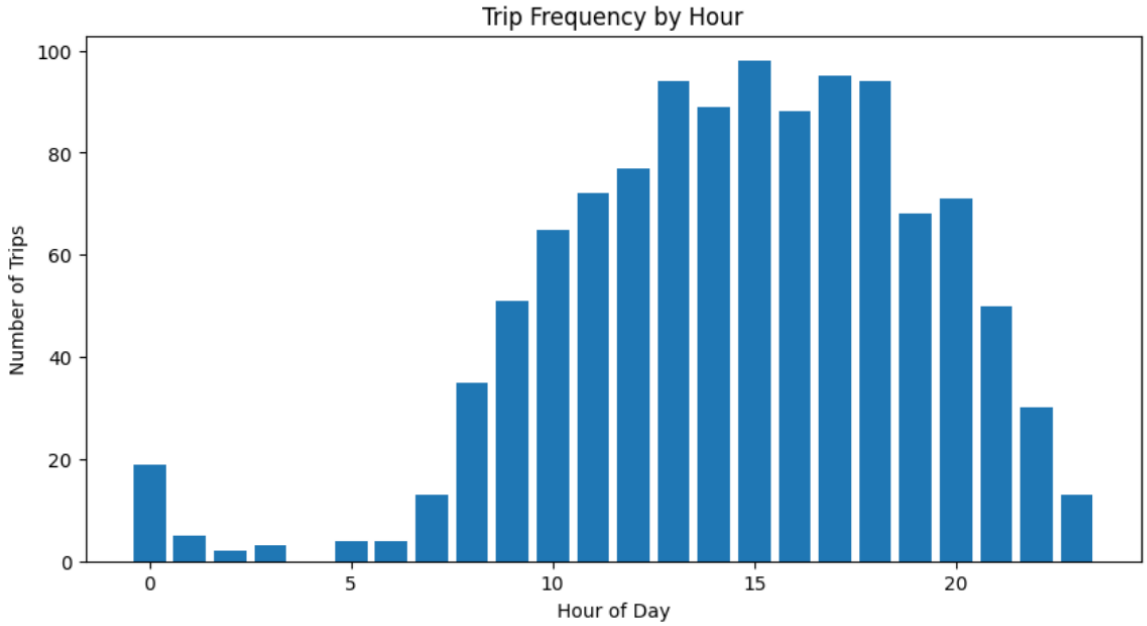
07:42 PM (5s) 20

```
#Visualization and Reporting
import matplotlib.pyplot as plt
import pandas as pd
#Trip Frequency by Hour
hour_df = df.groupBy("start_hour").count().orderBy("start_hour").toPandas()

plt.figure(figsize=(10,5))
plt.bar(hour_df["start_hour"], hour_df["count"])
plt.title("Trip Frequency by Hour")
plt.xlabel("Hour of Day")
plt.ylabel("Number of Trips")
plt.show()
```

(5) Spark Jobs

hour_df: pandas.core.frame.DataFrame = [start_hour: int32, count: int64]



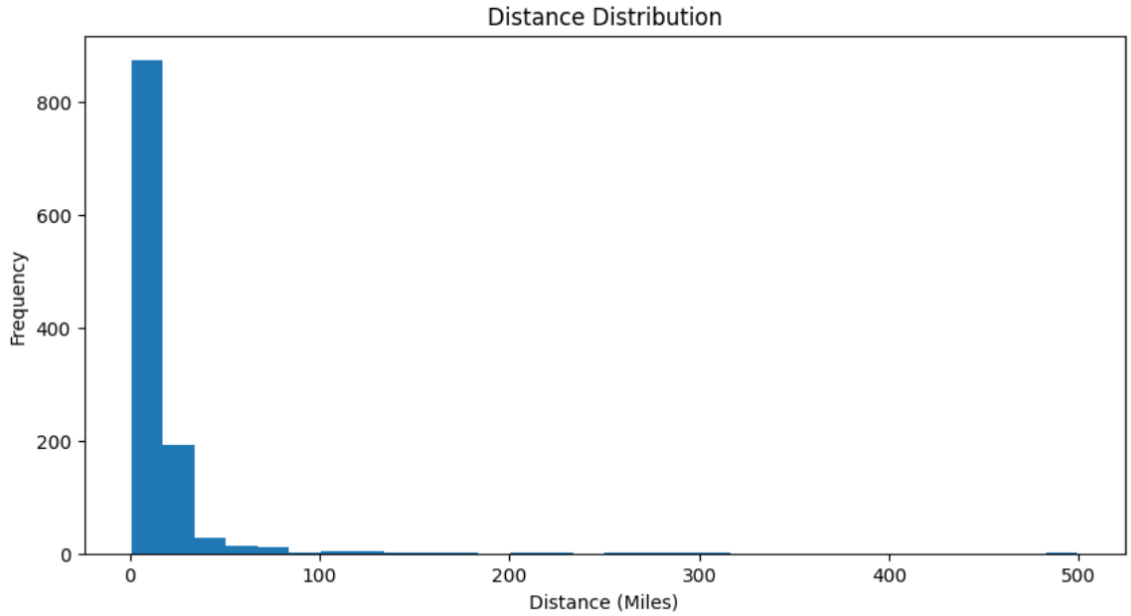
Distance Distribution

07:42 PM (1s) 21

```
#Distance Distribution
distance_df = df.select("MILES").dropna().toPandas()

plt.figure(figsize=(10,5))
plt.hist(distance_df["MILES"], bins=30)
plt.title("Distance Distribution")
plt.xlabel("Distance (Miles)")
plt.ylabel("Frequency")
plt.show()
```

(2) Spark Jobs



Distance vs Duration Relationship

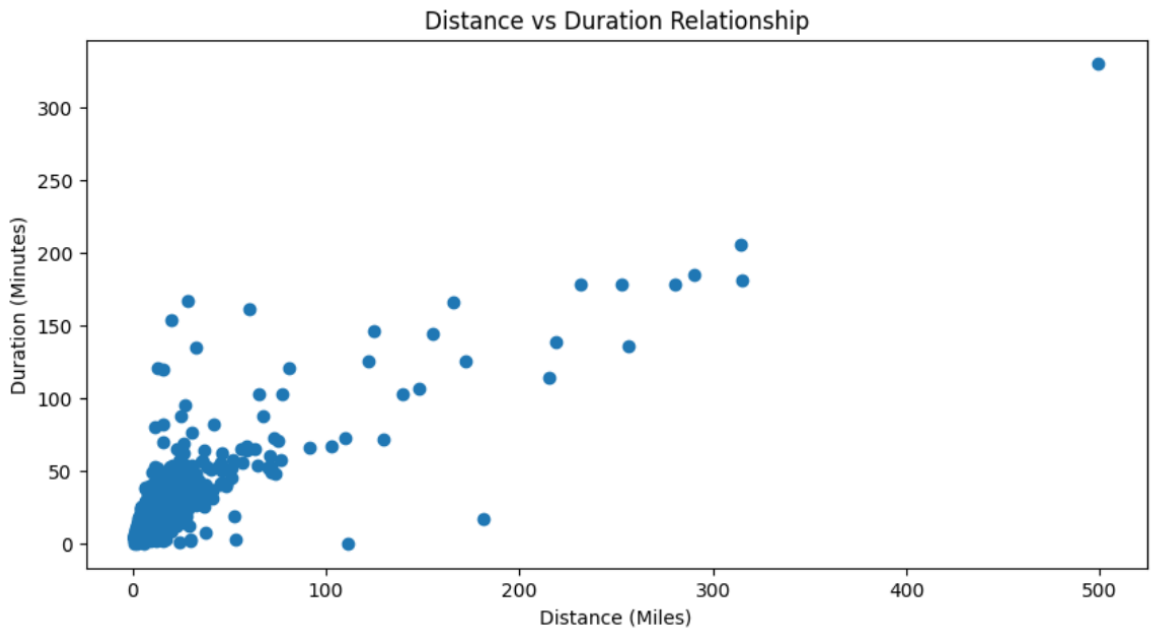
07:42 PM (1s) 22

```
#Distance vs Duration Relationship
scatter_df = df.select("MILES", "duration").dropna().toPandas()

plt.figure(figsize=(10,5))
plt.scatter(scatter_df["MILES"], scatter_df["duration"])
plt.title("Distance vs Duration Relationship")
plt.xlabel("Distance (Miles)")
plt.ylabel("Duration (Minutes)")
plt.show()
```

(2) Spark Jobs

scatter_df: pandas.core.frame.DataFrame = [MILES: float64, duration: float64]



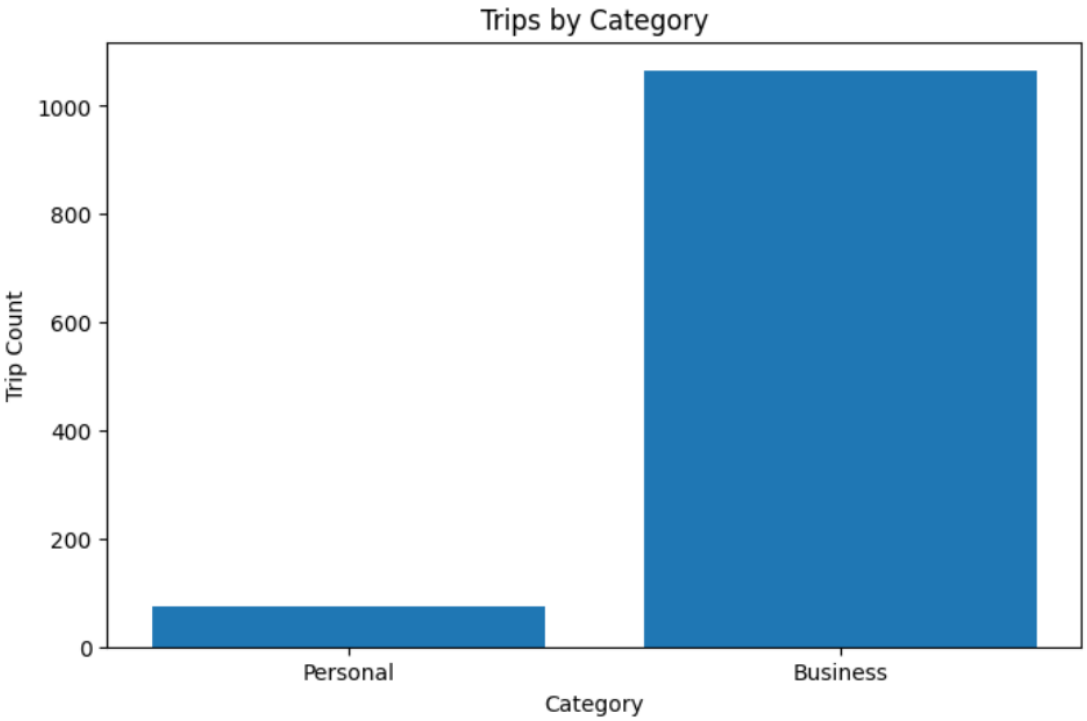
Trips by Category

07:42 PM (1s) 23

```
#Trips by Category
category_df = df.groupBy("CATEGORY").count().toPandas()

plt.figure(figsize=(8,5))
plt.bar(category_df["CATEGORY"], category_df["count"])
plt.title("Trips by Category")
plt.xlabel("Category")
plt.ylabel("Trip Count")
plt.show()
```

category_df: pandas.core.frame.DataFrame = [CATEGORY: object, count: int64]



Trips by Day of the Week

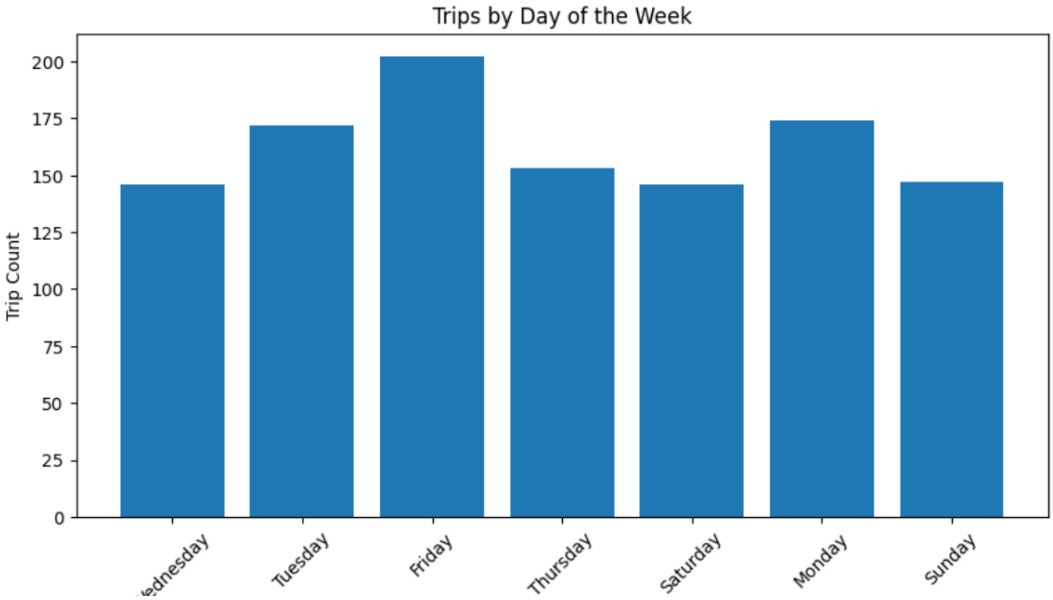
07:42 PM (1s) 24

```
#Trips by Day of the Week
day_df = df.groupBy("day").count().toPandas()

plt.figure(figsize=(10,5))
plt.bar(day_df["day"], day_df["count"])
plt.title("Trips by Day of the Week")
plt.xlabel("Day")
plt.ylabel("Trip Count")
plt.xticks(rotation=45)
plt.show()
```

(3) Spark Jobs

day_df: pandas.core.frame.DataFrame = [day: object, count: int64]



09:17 AM (6s) 25 Python

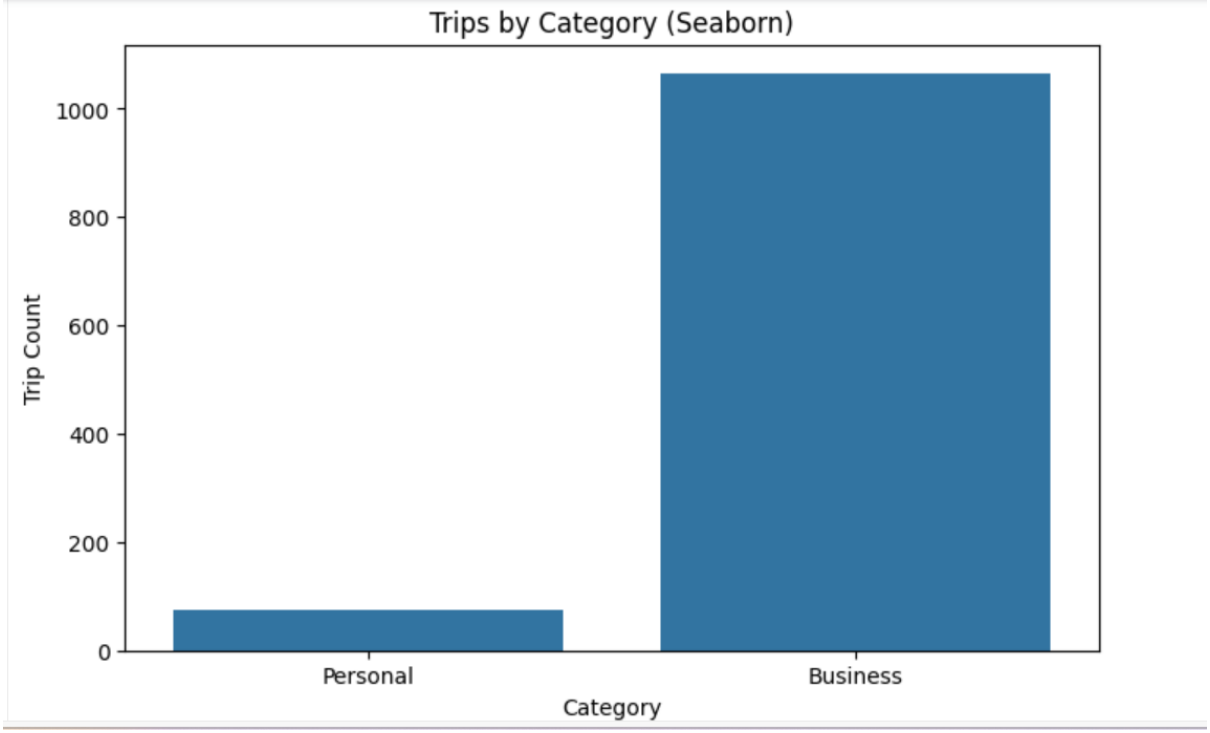
```python
# Seaborn Visualization

import seaborn as sns
import matplotlib.pyplot as plt

# Convert category trip counts to pandas
category_df = df.groupBy("CATEGORY").count().toPandas()

plt.figure(figsize=(8,5))
sns.barplot(data=category_df, x="CATEGORY", y="count")
plt.title("Trips by Category (Seaborn)")
plt.xlabel("Category")
plt.ylabel("Trip Count")
plt.show()
```

(3) Spark Jobs



5. Tools and Technologies

The following technologies are used in this big data analytics workflow:

- Microsoft Azure
- Azure Data Lake Storage
- Azure Databricks
- Apache Spark
- PySpark
- Python
- Matplotlib
- Seaborn

These tools enable scalable storage, distributed processing, and effective data visualization.

8. Conclusion

Big data analytics using PySpark on Azure provides a scalable platform for processing and analyzing large datasets. By applying distributed data processing techniques, the

system can efficiently analyze travel patterns, identify inefficiencies, and detect anomalies.

The integration of Azure cloud infrastructure with Spark processing and Python visualization tools enables efficient data analysis and reporting. This approach allows organizations to transform raw trip data into actionable insights that support operational improvements and strategic planning.